

可能耗時較長的函式分析報告

分析日期: 2025-10-03 分析範圍: 系統核心模組中可能耗時數秒到數十秒的函式 耗時標準: ⚠ 1-5 秒 | 5-30秒 | 30秒以上

執行摘要

本報告識別出 **15 個高耗時函式**, 分布在 5 個核心模組中。主要耗時來源包括: - **LLM 調用** (最高可達 120 秒) - **Embedding 模型載入與向量化** (首次載入 5-15 秒) - **Milvus 向量搜尋** (取決於資料量, 通常 0.5-5 秒) - **DuckDB 複雜查詢** (大量資料時 1-10 秒)

極高耗時函式 (30秒以上)

1. MGFDKernel.py

`llm_initializer.safe_completion()` 調用

- 檔案位置: [libs/MGFDKernel.py:1043-1047](#)
- 預估耗時: 5-120 秒
- 耗時原因:
 - LLM 推理運算密集
 - 明確設定 `timeout=120` 秒
 - 生成 2048 token 的回應
 - 網路延遲 (如果是遠端 LLM 服務)
 - 觸發條件: 每次使用者查詢且有產品資料時
- 程式碼片段:

```
# Line 1043-1047
llm_output = await asyncio.wait_for(
    asyncio.to_thread(self.llm_initializer.safe_completion, current_prompt, 2048),
    timeout=120 # 明確設定 120 秒超時
)
```

- 優化建議:
 - 使用串流輸出 (streaming) 提升使用者體驗
 - 實作 Redis 快取常見查詢
 - 考慮使用較小的 token 限制
 - 加入 loading 進度指示

`get_query_rule_from_user_query()`

- 檔案位置: [libs/MGFDKernel.py:580-607](#)
- 預估耗時: 5-120 秒

- **耗時原因:**
- 同樣調用 `llm_initializer.safe_completion()`
- 用於解析使用者意圖和實體
- 有 120 秒超時保護
- **觸發條件:** 每次產品搜尋請求
- **程式碼片段:**

```
# Line 583-586
self.query_rule = await asyncio.wait_for(
    asyncio.to_thread(self.llm_initializer.safe_completion, qry_str, 2048),
    timeout=120,
)
```

- **優化建議:**
- 快取常見查詢模式的解析結果
- 使用更輕量的意圖識別模型
- 並行處理 (與其他操作同時執行)

高耗時函式 (5-30秒)

2. KnowledgeManageHandler/knowledge_manager.py

`_initialize_ai_components()`

- **檔案位置:** [libs/KnowledgeManageHandler/knowledge_manager.py:161-212](#)
- **預估耗時:** 5-15 秒 (首次載入)
- **耗時原因:**
- SentenceTransformer 模型下載與載入 (首次)
- Milvus 連接與 Collection 載入
- 模型權重檔案可能數百 MB
- **觸發條件:** 系統初始化時 (一次性)
- **程式碼片段:**

```
# Line 164-167
embedding_model = 'sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2'
self.sentence_transformer = SentenceTransformer(embedding_model)
```

- **優化建議:**
- 預先下載模型到本地
- 使用模型快取機制
- 延遲載入 (lazy initialization)
- 使用較小的 embedding 模型

`search_product_data()`

- **檔案位置:** [libs/KnowledgeManageHandler/knowledge_manager.py:1676-1827](#)

- 預估耗時: 2-10 秒
- 耗時原因:
 - Milvus 語義搜尋 (top_k=30)
 - 產品代碼驗證 (_validate_modeltype_exists)
 - DuckDB IN 查詢 (可能涉及大量 modeltype)
 - 多個步驟的串行執行
- 觸發條件: 每次產品搜尋請求
- 程式碼片段:

```
# Line 1697-1701
semantic_results = self.milvus_semantic_search(
    query_text=enhanced_query,
    top_k=30 # 搜尋 30 個結果
)
```

- 優化建議:
 - 並行執行 Milvus 搜尋和產品代碼檢測
 - 減少 top_k 數量
 - 為 DuckDB 查詢建立索引
 - 快取熱門查詢結果

milvus_semantic_search()

- 檔案位置: libs/KnowledgeManageHandler/knowledge_manager.py:807-900
- 預估耗時: 1-5 秒
- 耗時原因:
 - Sentence embedding 向量化
 - Milvus 向量搜尋 (取決於 Collection 大小)
 - 結果格式化與排序
- 觸發條件: 所有語義搜尋請求
- 程式碼片段:

```
# Line 842
query_vector = self.sentence_transformer.encode(query_text).tolist()

# Line 870-877
results = self.milvus_query.collection.search(
    data=[query_vector],
    anns_field="embedding",
    param=search_params,
    limit=top_k,
    output_fields=output_fields,
    expr=filter_expr
)
```

- 優化建議:
 - 快取常見查詢的 embedding
 - 調整 Milvus 搜尋參數 (nprobe)

- 使用 ANN 索引優化
- 批次處理多個查詢

`encode_text()` 和 `encode_texts()`

- 檔案位置:
- [libs/KnowledgeManageHandler/knowledge_manager.py:1168-1180](#)
- [libs/KnowledgeManageHandler/knowledge_manager.py:1182-1194](#)
- 預估耗時: 0.5-3 秒 (取決於文字長度)
- 耗時原因:
- Transformer 模型推理
- 批次處理多個文本時更耗時
- 觸發條件: 每次需要文本向量化時
- 優化建議:
- GPU 加速
- 批次處理優化
- 使用量化模型

⚠ 中等耗時函式 (1-5秒)

3. MGFDKernel.py

`_process_message_internal()`

- 檔案位置: [libs/MGFDKernel.py:940-1083](#)
- 預估耗時: ⚠ 2-10 秒 (總計)
- 耗時原因:
- 串行執行多個耗時操作:
 1. 關鍵詞解析
 2. LLM 查詢規則生成 (5-120秒)
 3. 知識庫搜尋 (2-10秒)
 4. 產品資料後處理
 5. LLM 生成回應 (5-120秒)
- 觸發條件: 每次使用者訊息處理
- 優化建議:
- 並行執行獨立操作
- 使用 progressive streaming 提供即時回饋
- 實作快取機制

`_postprocess_product_data()`

- 檔案位置: [libs/MGFDKernel.py:343-473](#)
- 預估耗時: ⚠ 1-3 秒
- 耗時原因:
- 複雜的產品排序演算法
- 特徵比對與評分 (`feature_score_and_hits`)

- 多個 regex 檢查
- 處理大量產品時 (max_products=5-10)
- 觸發條件: 每次產品查詢有結果時
- 程式碼片段:

Line 366-413: 複雜的評分函數

```
def feature_score_and_hits(prod: Dict[str, Any]) -> (int, List[str]):
    total = 0
    hits: List[str] = []
    features = (self.nb_feature_table or {}).get("features", [])
    # ... 多重迴圈和 regex 檢查
```

- 優化建議:
- 預先編譯 regex patterns
- 使用向量化操作
- 限制處理的產品數量
- 並行處理產品評分

4. UserInputHandler/UserInputHandler.py

getEntityParsingPrompt()

- 檔案位置: [libs/UserInputHandler/UserInputHandler.py:162-277](#)
- 預估耗時: ⚠ 微小 (僅字串處理)
- 備註: 本身不耗時,但其結果會傳給 LLM 造成延遲

parse()

- 檔案位置: [libs/UserInputHandler/UserInputHandler.py:281-338](#)
- 預估耗時: ⚠ 0.5-2 秒
- 耗時原因:
 - 多個 async 函數串行調用
 - 意圖分類 + 槽位抽取 + 控制邏輯判斷
 - 多個 regex 匹配操作
- 觸發條件: 每次輸入解析
- 優化建議:
 - 並行執行獨立操作
 - 快取常見模式
 - 預先編譯 regex

5. services/milvus_service.py

get_collection_data()

- 檔案位置: [libs/services/milvus_service.py:198-307](#)
- 預估耗時: ⚠ 1-5 秒
- 耗時原因:
 - Collection 載入 (如果未載入)

- 大量資料查詢 (query_limit 最高 16384)
- 分頁處理
- 資料格式化
- 觸發條件: Milvus 資料瀏覽器查看資料時
- 程式碼片段:

```
# Line 246
query_limit = min(offset + limit, 16384) # Milvus 查詢限制

# Line 267-271
results = collection.query(
    expr=expr,
    output_fields=output_fields,
    limit=query_limit
)
```

- 優化建議:
- 減少 query_limit
- 使用索引優化查詢
- 實作更高效的分頁機制

get_collections()

- 檔案位置: [libs/services/milvus_service.py:92-135](#)
- 預估耗時: ⚠ 0.5-3 秒
- 耗時原因:
- 列舉所有 collections
- 對每個 collection 獲取統計資訊
- 檢查索引狀態
- 觸發條件: 系統狀態檢查或 UI 顯示
- 優化建議:
- 快取 collection 列表
- 並行獲取統計資訊

6. RAG/DB/MilvusQuery.py

set_collection()

- 檔案位置: [libs/RAG/DB/MilvusQuery.py:28-41](#)
- 預估耗時: ⚠ 1-5 秒
- 耗時原因:
- Collection 載入到記憶體
- 索引載入
- 取決於 Collection 大小
- 觸發條件: 初始化或切換 Collection 時
- 程式碼片段:

```
# Line 32-36
if utility.has_collection(collection_name):
```

```
self.collection = Collection(collection_name)
self.collection.load() # 載入 Collection 到記憶體
```

- 優化建議:
- 延遲載入
- 保持 Collection 常駐記憶體
- 使用連接池

search()

- 檔案位置: [libs/RAG/DB/MilvusQuery.py:43-90](#)
- 預估耗時: ⚠ 0.5-3 秒
- 耗時原因:
- Query embedding 生成
- 向量搜尋
- 結果格式化
- 觸發條件: 每次向量搜尋
- 優化建議:
- 調整搜尋參數
- 快取常見 embeddings
- 批次處理

7. RAG/DB/AsyncDuckDBQuery.py

execute_async()

- 檔案位置: [libs/RAG/DB/AsyncDuckDBQuery.py:155-228](#)
- 預估耗時: ⚠ 0.5-10 秒 (取決於查詢複雜度)
- 耗時原因:
- 複雜 SQL 查詢
- 大量資料掃描
- Thread pool 執行開銷
- 觸發條件: 所有 async DuckDB 查詢
- 程式碼片段:

```
# Line 191-201
rows, columns, exec_time = await asyncio.wait_for(
    loop.run_in_executor(
        self.executor,
        self._execute_sync,
        query,
        parameters,
        fetch_mode
    ),
    timeout=query_timeout # 預設 60 秒
)
```

- 優化建議:

- 建立索引
- 優化查詢語句
- 使用查詢快取

query_by_modeltypes()

- 檔案位置: [libs/RAG/DB/AsyncDuckDBQuery.py:230-278](#)
- 預估耗時: ⚠ 1-5 秒
- 耗時原因:
 - IN 查詢 (多個 modeltype)
 - 全表掃描 (如果沒有索引)
- 觸發條件: 批次查詢產品規格
- 優化建議:
 - 為 modeltype 建立索引
 - 限制查詢數量
 - 使用查詢快取

8. RAG/DB/DuckDBQuery.py

query() 和 query_with_params()

- 檔案位置:
 - [libs/RAG/DB/DuckDBQuery.py:18-26](#)
 - [libs/RAG/DB/DuckDBQuery.py:28-36](#)
- 預估耗時: ⚠ 0.5-10 秒 (取決於查詢)
- 耗時原因:
 - 同步 I/O 阻塞
 - 複雜查詢掃描大量資料
- 優化建議:
 - 改用 AsyncDuckDBQuery
 - 建立索引
 - 優化查詢語句

函式耗時排名 (Top 10)

排名	函式名稱	檔案	預估耗時	優先級
1	llm_initializer.safe_completion()	MGFDKernel.py	5-120s	極高
2	get_query_rule_from_user_query()	MGFDKernel.py	5-120s	極高
3	_initialize_ai_components()	knowledge_manager.py	5-15s	中 (一次性)
4	search_product_data()	knowledge_manager.py	2-10s	高
5	milvus_semantic_search()	knowledge_manager.py	1-5s	高
6	_process_message_internal()	MGFDKernel.py	⚠ 2-10s	高
7	_postprocess_product_data	MGFDKernel.py	⚠ 1-3s	中

排名	函式名稱	檔案	預估耗時	優先級
	()			
8	encode_text/ encode_texts()	knowledge_manager.py	⚠ 0.5-3s	中
9	get_collection_data()	milvus_service.py	⚠ 1-5s	低 (管理功能)
10	execute_async()	AsyncDuckDBQuery.py	⚠ 0.5-10s	中

優化建議總結

立即執行 (高優先級)

1. 實作 Progressive Streaming

2. 對 LLM 回應使用串流輸出
3. 提供即時進度回饋

4. 改善使用者體驗

5. 快取機制

6. Redis 快取 LLM 常見回應
7. 快取常見查詢的 embeddings

8. 快取熱門產品規格

9. 並行處理

10. Milvus 搜尋 + 產品代碼檢測並行
11. 多個獨立 DuckDB 查詢並行執行
12. 特徵評分並行化

中期執行 (中優先級)

1. 資料庫優化

2. DuckDB 的 modeltype 欄位建立索引
3. Milvus 調整 nprobe 參數

4. 優化查詢語句

5. 模型優化

6. 考慮使用更小的 embedding 模型
7. GPU 加速 (如果有硬體支援)

8. 模型量化

9. 程式碼優化

10. 預先編譯 regex patterns
11. 減少不必要的資料轉換

12. 使用更高效的資料結構

長期執行 (低優先級)

1. 架構優化

2. 引入訊息佇列 (如 RabbitMQ)
3. 實作微服務架構
4. 使用 CDN 快取靜態資源

5. 監控與分析

6. 加入效能監控 (APM)
7. 記錄每個函式的實際執行時間
8. 建立效能儀表板

監控建議

為了更精確地評估函式耗時,建議實作以下監控:

1. 函式級別的時間追蹤

```
import time
import logging
from functools import wraps

def track_time(func):
    @wraps(func)
    async def async_wrapper(*args, **kwargs):
        start = time.time()
        try:
            result = await func(*args, **kwargs)
            duration = time.time() - start
            logging.info(f"{func.__name__} completed in {duration:.3f}s")
            return result
        except Exception as e:
            duration = time.time() - start
            logging.error(f"{func.__name__} failed after {duration:.3f}s: {e}")
            raise

    @wraps(func)
    def sync_wrapper(*args, **kwargs):
        start = time.time()
        try:
            result = func(*args, **kwargs)
            duration = time.time() - start
            logging.info(f"{func.__name__} completed in {duration:.3f}s")
            return result
        except Exception as e:
            duration = time.time() - start
            logging.error(f"{func.__name__} failed after {duration:.3f}s: {e}")
```

```

        raise

    if asyncio.iscoroutinefunction(func):
        return async_wrapper
    return sync_wrapper

# 使用範例
@track_time
async def search_product_data(self, message: str):
    # ... 函式實作

```

2. 關鍵指標收集

- LLM 調用次數與平均耗時
- Milvus 搜尋次數與平均耗時
- DuckDB 查詢次數與平均耗時
- 快取命中率
- 並行執行效率

3. 效能告警

設定耗時閾值告警: - LLM 調用 > 30 秒 - Milvus 搜尋 > 5 秒 - DuckDB 查詢 > 5 秒 - 整體請求處理 > 15 秒

測試建議

建議建立以下測試腳本來驗證優化效果:

```

# benchmark_script.py
import asyncio
import time
from libs.MGFDKernel import MGFDKernel
from libs.KnowledgeManageHandler.knowledge_manager import KnowledgeManager

async def benchmark_search():
    """基準測試：產品搜尋"""
    km = KnowledgeManager()

    test_queries = [
        "推薦遊戲筆電",
        "輕薄便攜筆電",
        "819 系列規格",
        "比較 APX819 和 APX839"
    ]

    results = {}
    for query in test_queries:
        start = time.time()

```

```

        result = km.search_product_data(query)
        duration = time.time() - start
        results[query] = duration
        print(f"{query}: {duration:.3f}s")

    return results

async def benchmark_llm():
    """基準測試: LLM 調用"""
    kernel = MGFDKernel()

    test_prompts = [
        "簡單查詢",
        "複雜多條件查詢",
        "比較型查詢"
    ]

    # ... 測試實作

if __name__ == "__main__":
    asyncio.run(benchmark_search())

```

結論

系統中確實存在多個耗時較長的函式,主要集中在:

1. LLM 相關操作 (最主要瓶頸,5-120 秒)
2. 向量化與語義搜尋 (1-5 秒)
3. 資料庫查詢 (0.5-10 秒)

關鍵優化方向: - ✓ 實作 progressive streaming 提升體驗 - ✓ 加入多層快取機制 - ✓ 盡可能並行執行獨立操作 - ✓ 優化資料庫查詢與索引 - ✓ 建立效能監控與告警

透過這些優化,預計可以將平均請求處理時間從目前的 **10-30 秒** 降低到 **3-8 秒**,並在快取命中時進一步降低到 **1-2 秒**。

報告產生時間: 2025-10-03 **分析工具:** Claude Code (SuperClaude Framework) **分析者:** AI Assistant